

Yenepoya (Deemed to be University)

Low Level Design (LLD)

on

**Honeypot Network for Agriculture Threat
Intelligence**

Student Name: Farhan N Methar

Project ID:PRJN26-205

**Industry
Mentor:**

SHASHANK

Guided by: SUZANA SHAROL

Table of Contents

- 1. Introduction.**
 - 1.1. **Scope of the document.**
 - 1.2. **Intended Audience**
 - 1.3. **System overview.**
- 2. Module Design**
 - 2.1. **Module: Server Initialization**
 - 2.2. **Module: handle client () Function**
 - 2.3. **Module: IP Blocking & Persistence**
 - 2.4. **Module: CSV Logging**
 - 2.5. **Module: Threading & Lock**
- 3. Function-Level Design**
- 4. Data Design**
 - 4.1. **Data Model**
 - 4.2. **Data Access Mechanism**
 - 4.3. **Data Retention Policies**
- 5. Process Flow**
- 6. Error Handling**
- 7. Non-Functional Requirements**
 - 7.1. **Security Aspects**
 - 7.2. **Performance Aspects**
- 8. References**

1. Introduction

1.1 Scope of the Document

This document provides a detailed low-level description of the Honeypot Network for Agriculture Threat Intelligence system. It covers the function-level design, module breakdown, logic flow, and data handling of each component implemented in Python using the Socket library.

1.2 Intended Audience

This document is intended for project evaluators, technical reviewers, and instructors who want to understand the internal working of each module in the honeypot system.

1.3 System Overview

The system is a Python-based honeypot that simulates an Agri Sense IoT Gateway on port 8888. It monitors incoming TCP connections, logs attacker details such as IP address, port number, timestamp, and number of attempts into a CSV file. IPs that exceed 5 connection attempts are marked as Suspicious and permanently blocked. The system uses threading to handle multiple simultaneous connections and a Lock mechanism to prevent race conditions and data corruption.

2. Module Design

The system is divided into 5 functional modules:

#	Module Name	Description
1	Server Initialization	Creates TCP socket, binds to port 8888, starts listening
2	handle client ()	Handles each connection: checks block list, counts attempts, logs data
3	IP Blocking & Persistence	Detects suspicious IPs, blocks them, saves to blocked_ips.txt
4	CSV Logging	Logs Time, IP, Port, Attempts, Status to log.csv
5	Threading & Lock	Handles multiple connections simultaneously with thread safety

2.1 Module: Server Initialization

This module sets up the TCP server using Python's socket library.

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("0.0.0.0", 8888))
server.listen(5)
```

Input: None

Output: Active server listening on port 8888

AF_INET means IPv4 protocol. SOCK_STREAM means TCP connection. SO_REUSEADDR allows port reuse after restart. listen (5) allows up to 5 queued connections.

2.2 Module: handle client() Function

This is the core function of the system. It is called in a new thread for every incoming connection.

```
def handle_client(client, addr):
    ip = addr[0]
    port = addr[1]
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with lock:
        # check block, count attempts, log data
        client.send("AgriSense IoT Gateway v2.1 - 403 Access Denied".encode())
        client.close()
```

Input: client socket object, addr tuple (IP, port)

Output: Logged entry in CSV, printed alert in terminal, fake response sent to attacker

2.3 Module: IP Blocking & Persistence

This module detects and blocks suspicious IPs. An IP is flagged as Suspicious after 5 or more connection attempts. The blocked IP is saved to blocked_ips.txt so it remains blocked even after the server restarts.

```
if attempts >= 5:
    status = "Suspicious"
    blocked_ips.add(ip)
    with open(BLOCKED_IPS_FILE, "a") as f:
        f.write(ip + "\n")
```

Input: IP address, attempt count

Output: IP added to blocked set and written to blocked_ips.txt

2.4 Module: CSV Logging

Every connection attempt is recorded in log.csv with the following fields: Time, IP Address, Port, Attempts, and Status.

```
writer.writerow([timestamp, ip, port, attempts, status])
```

Input: timestamp, ip, port, attempts, status

Output: One new row added to log.csv

2.5 Module: Threading & Lock

Threading allows the honeypot to handle multiple attackers simultaneously. The Lock prevents race conditions when two threads try to update shared data (Ip count, blocked_ips) at the same time.

```
lock = threading.Lock()

thread = threading.Thread(target=handle_client, args=(client, addr))
thread.start()

with lock:
    # shared data access here
```

Input: client socket, addr

Output: New thread created per connection, safe shared data access

3. Function-Level Design

Function	Input	Output	Description
<code>socket.socket()</code>	None	Server socket	Creates a TCP socket for the honey-pot
<code>server.bind()</code>	IP, Port	Bound socket	Assigns address 0.0.0.0:8888 to the server
<code>server.listen()</code>	Queue size	Listening server	Server starts accepting connections
<code>server.accept()</code>	None	client, addr	Accepts incoming connection from attacker
<code>handle_client()</code>	client, addr	Log entry, response	Core logic: block check, log, alert, respond
<code>threading.Thread()</code>	Function, args	Thread object	Creates a new thread per connection
<code>lock.acquire()</code>	None	Lock acquired	Prevents simultaneous shared data access
<code>csv.writer.writerow()</code>	Row data	CSV row	Writes one log entry to log.csv
<code>client.send()</code>	Message bytes	Response sent	Sends fake AgriSense denial message
<code>client.close()</code>	None	Connection closed	Closes connection after handling

4. Data Design

4.1 Data Model

The system stores all connection data in log.csv with the following structure:

Field	Type	Example	Description
Time	String	2026-04-10 15:44:01	Date and time of connection attempt
IP Address	String	127.0.0.1	IP address of the connecting client
Port	Integer	54321	Port number used by the attacker's connection
Attempts	Integer	5	Total number of attempts from this IP
Status	String	Suspicious	Normal if attempts < 5, Suspicious if >= 5

4.2 Data Access Mechanism

All data is stored in CSV format (log.csv). The file is appended to after each connection attempt. It can be opened in Microsoft Excel, LibreOffice Calc, or IBM Cognos for visualization and analysis.

4.3 Data Retention Policies

Logs are stored locally in log.csv and blocked_ips.txt. These files are persistent and survive server re-starts. There is no automatic deletion — files are retained for the duration of the project for analysis and reporting.

5. Process Flow

The following steps describe the complete flow of the system from startup to connection handling:

- Step 1: Server starts and binds to port 8888 on all interfaces (0.0.0.0)
- Step 2: Previously blocked IPs are loaded from blocked_ips.txt into memory
- Step 3: Server enters listening mode and waits for incoming connections
- Step 4: An incoming connection is accepted — server.accept() returns client socket and addr
- Step 5: A new thread is created to handle this connection via handle_client()
- Step 6: Inside handle_client(), a Lock is acquired to prevent race conditions
- Step 7: System checks if the IP is in blocked_ips — if yes, connection is dropped
- Step 8: If not blocked, attempt count for the IP is incremented in ip_count dictionary
- Step 9: If attempt count reaches 5 or more, IP is marked Suspicious and added to block list
- Step 10: Connection details are logged to log.csv (Time, IP, Port, Attempts, Status)
- Step 11: Lock is released. Fake response 'Agri Sense IoT Gateway v2.1 - 403 Access Denied' is sent
- Step 12: Connection is closed. Server returns to listening for the next connection

6. Error Handling

Scenario	Handling Method	Result
Server stopped by user (Ctrl+C)	KeyboardInterrupt exception caught	Server closes gracefully with message
IP connects more than 5 times	Threshold-based detection logic	IP blocked and saved to file
Server restarted	blocked_ips.txt loaded on startup	Previously blocked IPs remain blocked
Two connections at same time	Threading with Lock mechanism	Both handled safely without data corruption

7. Non-Functional Requirements

7.1 Security Aspects

- Detects and blocks unauthorized access attempts automatically
- Simulates a real AgriSense IoT Gateway to deceive and trap attackers
- Persistent IP blocking survives server restarts
- Thread-safe design prevents data corruption from simultaneous attacks
- Sends misleading fake response to confuse the attacker

7.2 Performance Aspects

- Lightweight system with minimal CPU and memory usage
- Threaded architecture allows handling multiple simultaneous connections
- CSV logging is fast and efficient for small-to-medium scale monitoring
- Lock mechanism adds minimal overhead while ensuring data integrity

8. References

- Python Official Documentation — <https://docs.python.org/3/library/socket.html>
- Python Threading Documentation — <https://docs.python.org/3/library/threading.html>
- Python CSV Documentation — <https://docs.python.org/3/library/csv.html>
- What is a Honeypot — <https://www.kaspersky.com/resource-center/threats/what-is-a-honeypot>
- IBM Cognos Analytics — <https://www.ibm.com/products/cognos-analytics>
- Network Security Concepts — Forouzan, B.A., Data Communications and Networking, McGraw-Hill